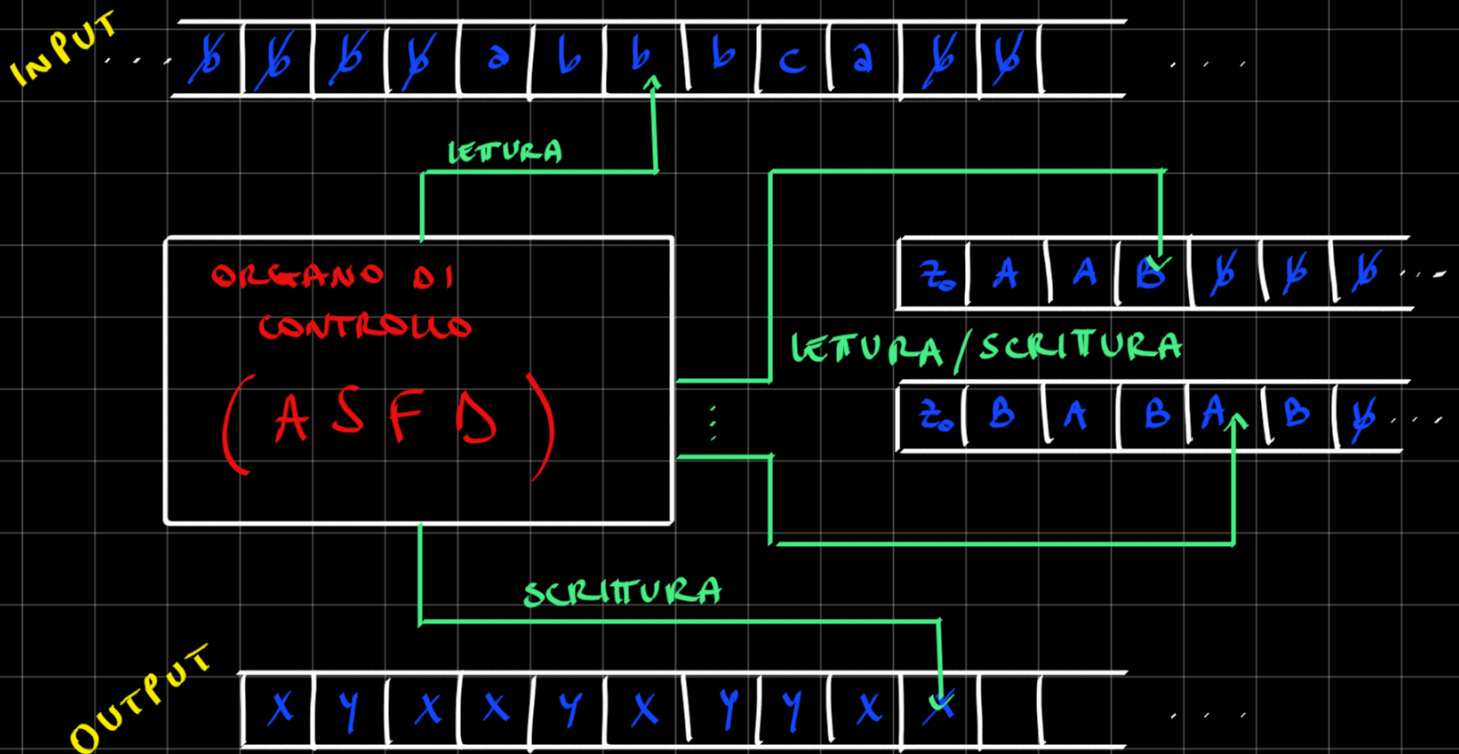


IMPORTANTE: Una macchina di Turing a k nastri ($k \in \mathbb{N}$), in realtà, ha $k+2$ nastri (dobbiamo includere anche quello di lettura e scrittura, oltre che quelli di memoria)

STRUTTURA:

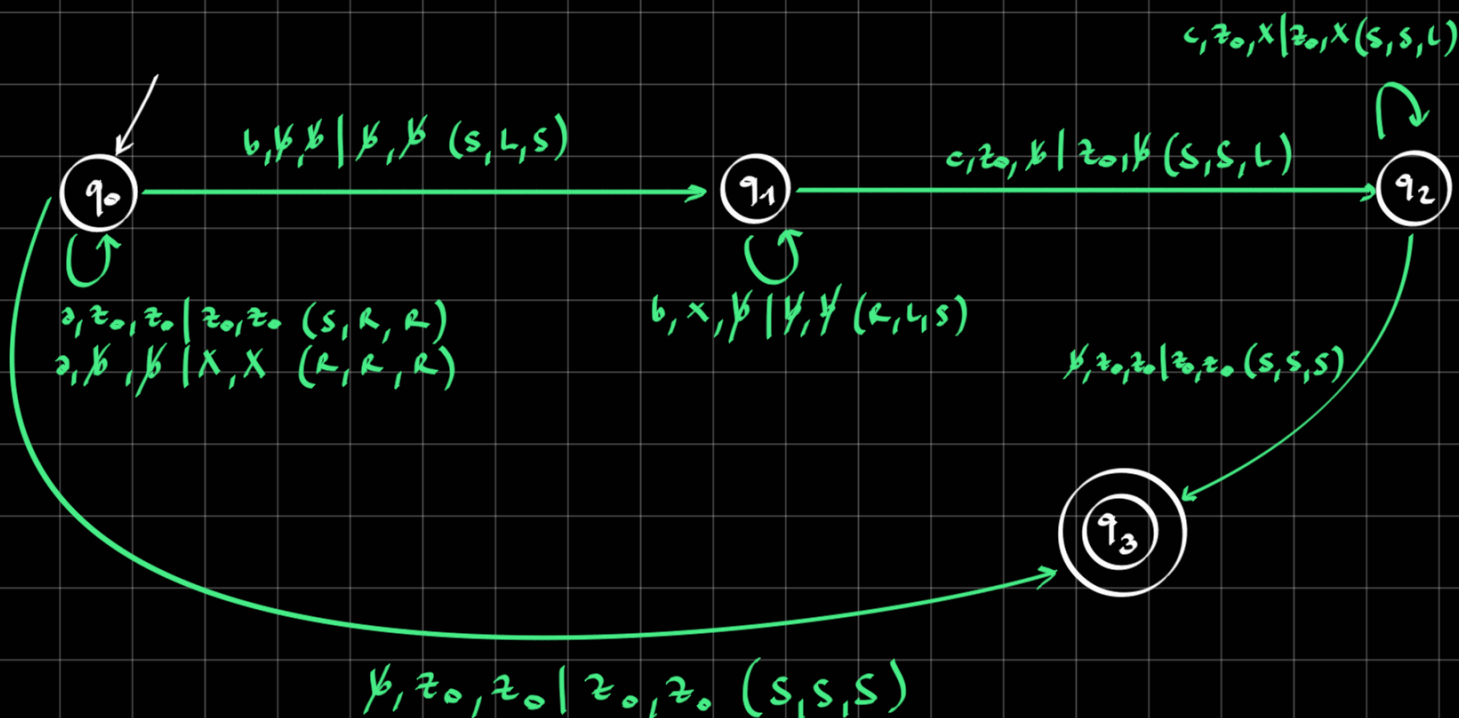


($\emptyset$: "blank", usato per riempire una casella del nastro con niente)

① Progettare una macchina di Turing che accetti:

$$L = \{ a^n b^n c^n \mid n \geq 0 \}$$

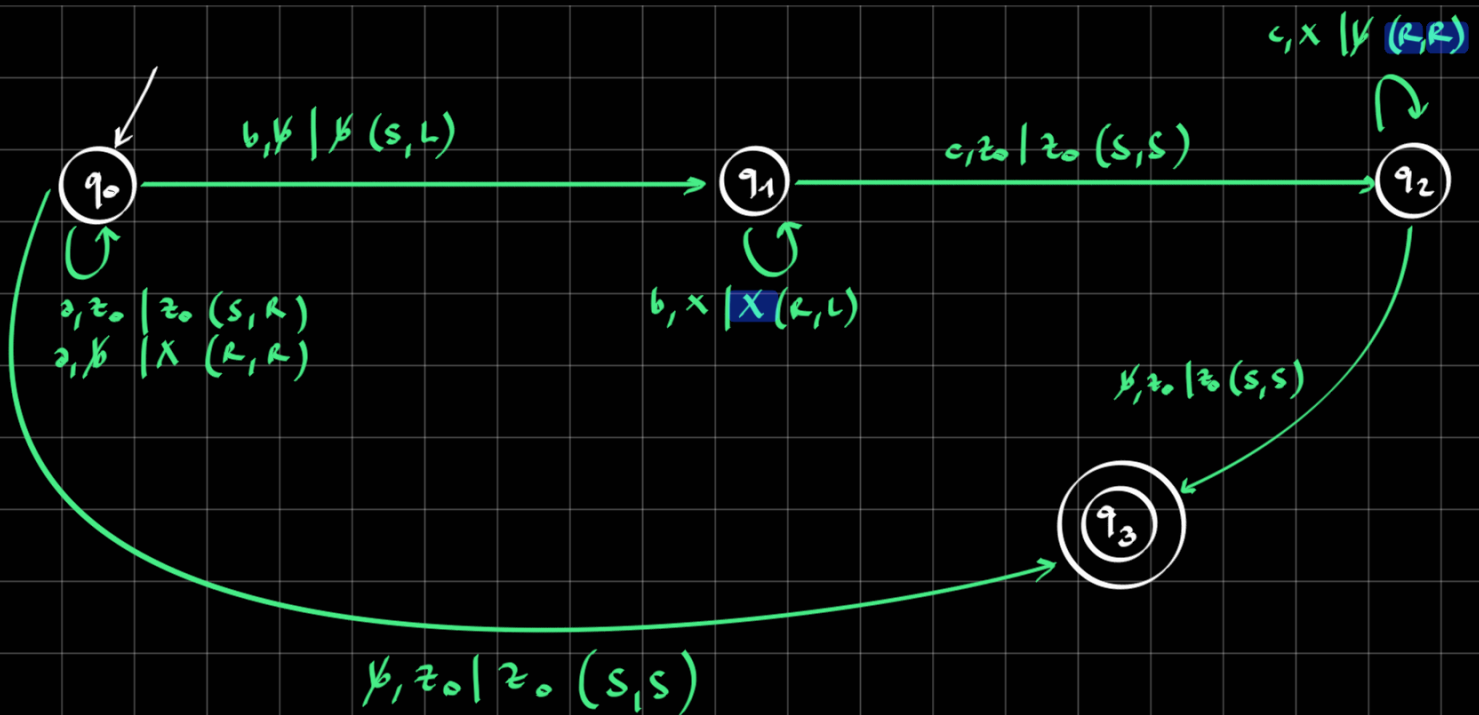
SOLUZIONE:



↑
Questa non è una
ε-MOSSA (non esistono
nelle macchine di Turing)
(l'importante è che non ci sia
non-determinismo!)

OSSERVA: si poteva anche fare con un solo marcatore:
nel conteggio delle 'b', sposta la testina
indietro senza cancellare niente...

SOLUZIONE ALTERNATIVA:



② $L = \{ a^{(2^n)} \mid n \geq 0 \}$

SOLUZIONE:

$$L = \{ a, aa, aaaa, (a)^8, (a)^{16}, \dots \}$$

Devo capire come memorizzare una potenza di 2 nel nastro e accettare una stringa con tale lunghezza. Ci sono due modi...

- ① Ricopro l'input in memoria. Rileggo l'input, ogni due caratteri, cancello un elemento dalla memoria. Alla fine verifico che il numero di

simboli in memoria sia pari. Ripeto fino a quando:

- Trovo un numero dispari di simboli in memoria (non accetto)
- Trovo un solo simbolo in memoria (accetto)

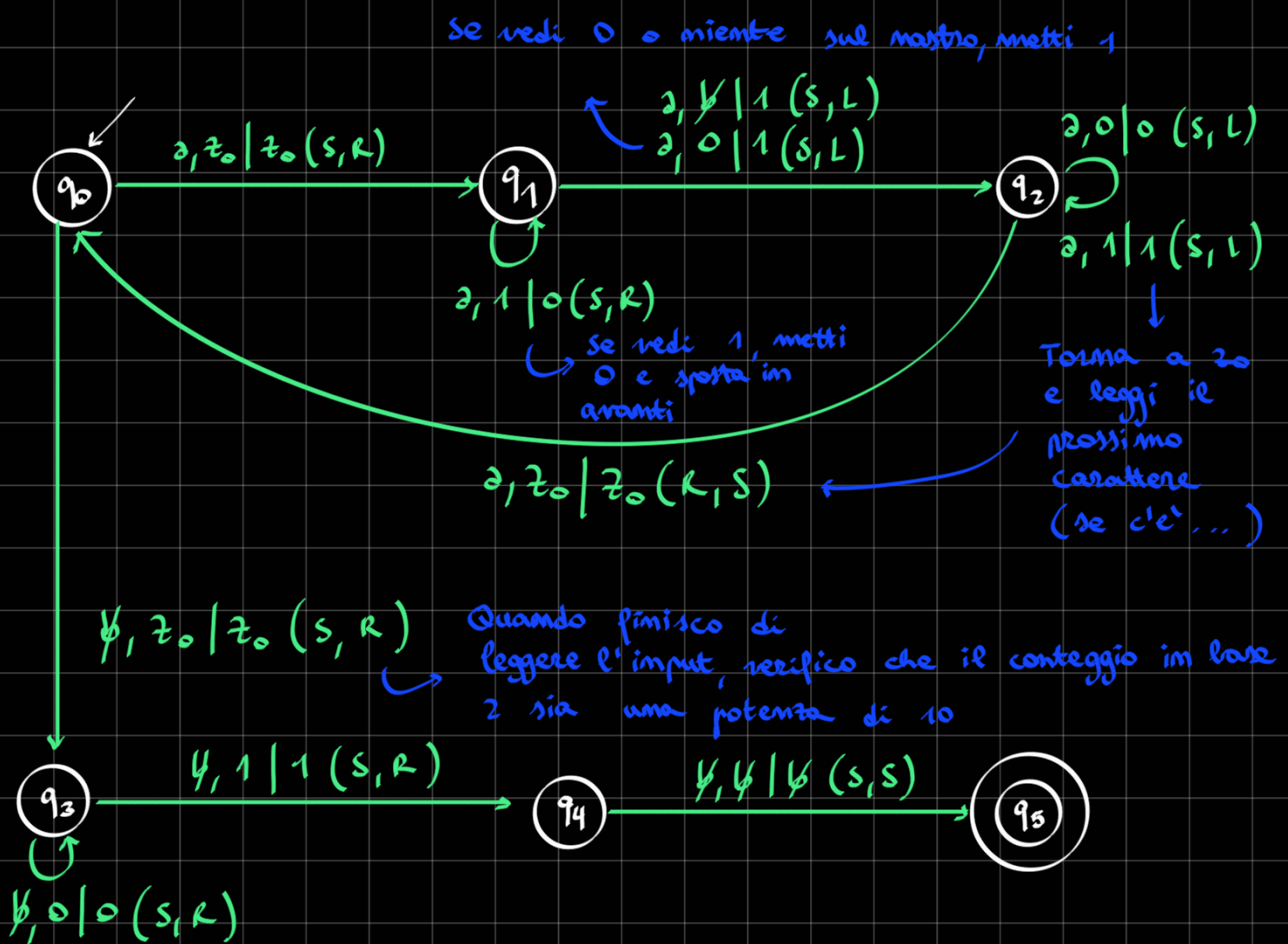
(Molto difficile da creare...)

2) Notiamo che una potenza di 2 in base 2 è un numero del tipo $1^1 0^n$. Quindi potrei contare in base 2 il numero di caratteri dell'input e verificare che il conteggio restituisca un numero di questo tipo

Nel nastro di memoria metteremo i simboli 1 e 0. A ogni carattere letto aggiungo 1 al numero in memoria

COME?

- Se vedo 0, quel simbolo in memoria diventa 1
- Se vedo 1, allora sostituisco 0 e mi sposto in avanti sul nastro e continuo a sommare 1 fino a quando non incontro uno zero



③ Progettare una macchina di Turing **A** **NASTRO SINGOLO** che riconosca il linguaggio precedente

SOLUZIONE:

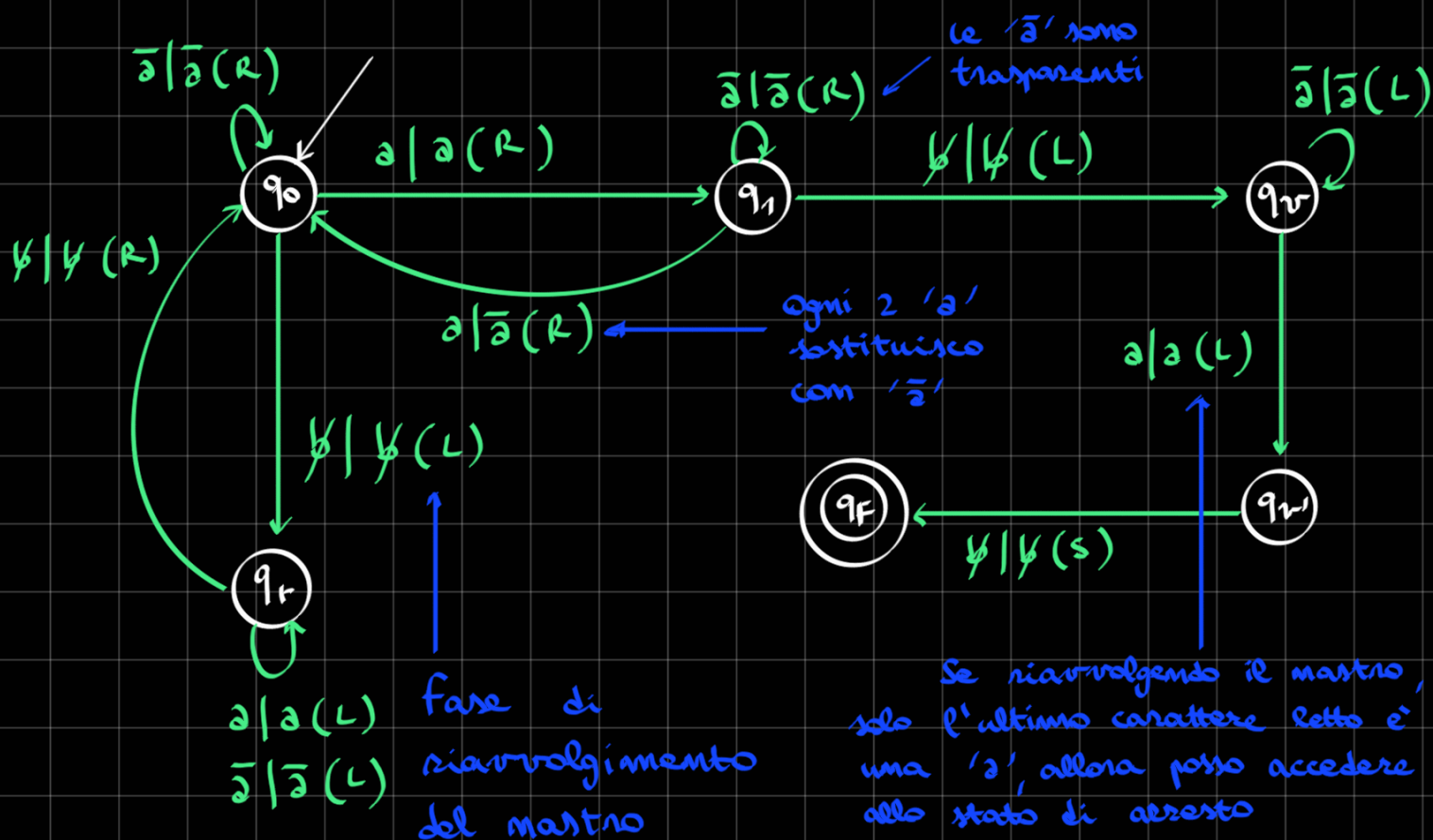
Posso usare un solo nastro per input, output e memoria

L'input si può anche cancellare, l'importante è arrivare con la macchina allo stato di arresto (cio che sta nel nastro non ha importanza...)

Una MT a maestro singolo ha la stessa "potenza" di una MT a un maestro di memoria, quindi le strategie scritte nell'es. prec. funzionano ugualmente, ma in questo caso è molto più impegnativo...

Si può progettare un altro algoritmo?

- Scorro da sinistra a destra. Ogni due 'a' sostituisco la 'a' presente con un simbolo 'ā'.
- Quando trovo blank, torno indietro e ripeto, considerando le 'ā' come trasparenti.
- Continuo fino a quando non incontro un blank dopo un numero dispari di 'a'. In tal caso ci sono due possibilità: la stringa non è accettabile oppure quella singola 'a' rimasta è la prima lettera della stringa (in questo caso la stringa è accettata).



CURIOSITÀ:

Mettiamo che la stringa in ingresso sia
lunga m

Quante transizioni vado a fare prima di bloccarmi (non deve essere un numero esatto, ma una stima dell'ordine di grandezza)?

Se $m = 2^n$, devo leggere per input da sinistra a destra e da destra a sinistra $n+1$ volte (il $+1$ è perché riscorro la stringa nella fase di verifica). Ogni volta che faccio una scansione da sinistra a destra e indietro, faccio $2m$ mosse. Infine, notiamo che $n = \log_2 m$

Quindi, il numero di mosse totali fatte è
circa $2m \log_2(m) + 1$

Questo valore ci dà una idea delle
COMPLESSITÀ dell'algoritmo...

$$\forall \epsilon > 0 \quad m \log(m) = \theta(m^{1+\epsilon})$$

X ESERCIZIO : (DIFFICILE)

Dimostra che il numero di mosse fatte dall'automa dell'esercizio (2) è tipicamente minore rispetto a quello dell'esercizio (3). Dunque la sua complessità è minore



logicamente ha senso: più
masteri hai, più lavori
velocemente